FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Michael Dominic L. Liggayu
Section:	An42
Professor:	Ms. Crisola Tan
Date:	September 30, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
   (Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)⇩
(Subject for change)

Source code:

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))


# -------- Agent Definition --------
class RandomWalker(ap.Agent):

    def setup(self):
        # Random starting position
        self.position = (
            random.randint(0, self.model.p.grid_size[0] - 1),
            random.randint(0, self.model.p.grid_size[1] - 1)
        )

        # Record the path
        self.path = [self.position]

    def step(self):
        # Possible movements
        directions = [
            (1, 0),    # Right
            (-1, 0),   # Left
            (0, 1),    # Up
```

```
            (0, -1)    # Down
        ]

        # Randomize the order of directions
        random.shuffle(directions)

        # Try each direction
        for direction in directions:

            x = self.position[0] + direction[0]
            y = self.position[1] + direction[1]

            # Check grid boundaries
            if (
                0 <= x < self.model.p.grid_size[0]
                and 0 <= y < self.model.p.grid_size[1]
            ):

                new_position = (x, y)

                # Check if another agent is there
                occupied = False

                for other in self.model.agents:
                    if other != self and other.position ==
new_position:
                        occupied = True
                        break

                # Move if the position is free
                if not occupied:
                    self.position = new_position
                    break

        # Record the new position
        self.path.append(self.position)


# -------- Model Definition --------
class RandomWalkModel(ap.Model):

    def setup(self):
        # Create agents
        self.agents = ap.AgentList(
            self,
```

```
            self.p.agents,
            RandomWalker
        )

    def step(self):
        # Move all agents
        for agent in self.agents:
            agent.step()


# -------- Parameters --------
parameters = {
    "agents": num_agents,
    "grid_size": (grid_size, grid_size),
    "steps": num_steps
}


# -------- Create Model --------
model = RandomWalkModel(parameters)
model.setup()


# -------- Create Animation --------
fig, ax = plt.subplots(figsize=(7, 7))

ax.set_xlim(-0.5, grid_size - 0.5)
ax.set_ylim(-0.5, grid_size - 0.5)

ax.set_xticks(range(grid_size))
ax.set_yticks(range(grid_size))

ax.grid(True)

ax.set_xlabel("X Position")
ax.set_ylabel("Y Position")
ax.set_title("Random Walk - Agent Movement")


# Get starting positions
x = [agent.position[0] for agent in model.agents]
y = [agent.position[1] for agent in model.agents]

# Create moving dots
scat = ax.scatter(x, y, s=100)
```

```
# Create path lines
lines = []

for agent in model.agents:
    line, = ax.plot([], [])
    lines.append(line)


# -------- Update Animation --------
def update(frame):

    # Move agents
    model.step()

    # Get new positions
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]

    # Move the dots
    scat.set_offsets(list(zip(x, y)))

    # Draw each agent's path
    for i, agent in enumerate(model.agents):

        path_x = [position[0] for position in agent.path]
        path_y = [position[1] for position in agent.path]

        lines[i].set_data(path_x, path_y)

    return [scat] + lines

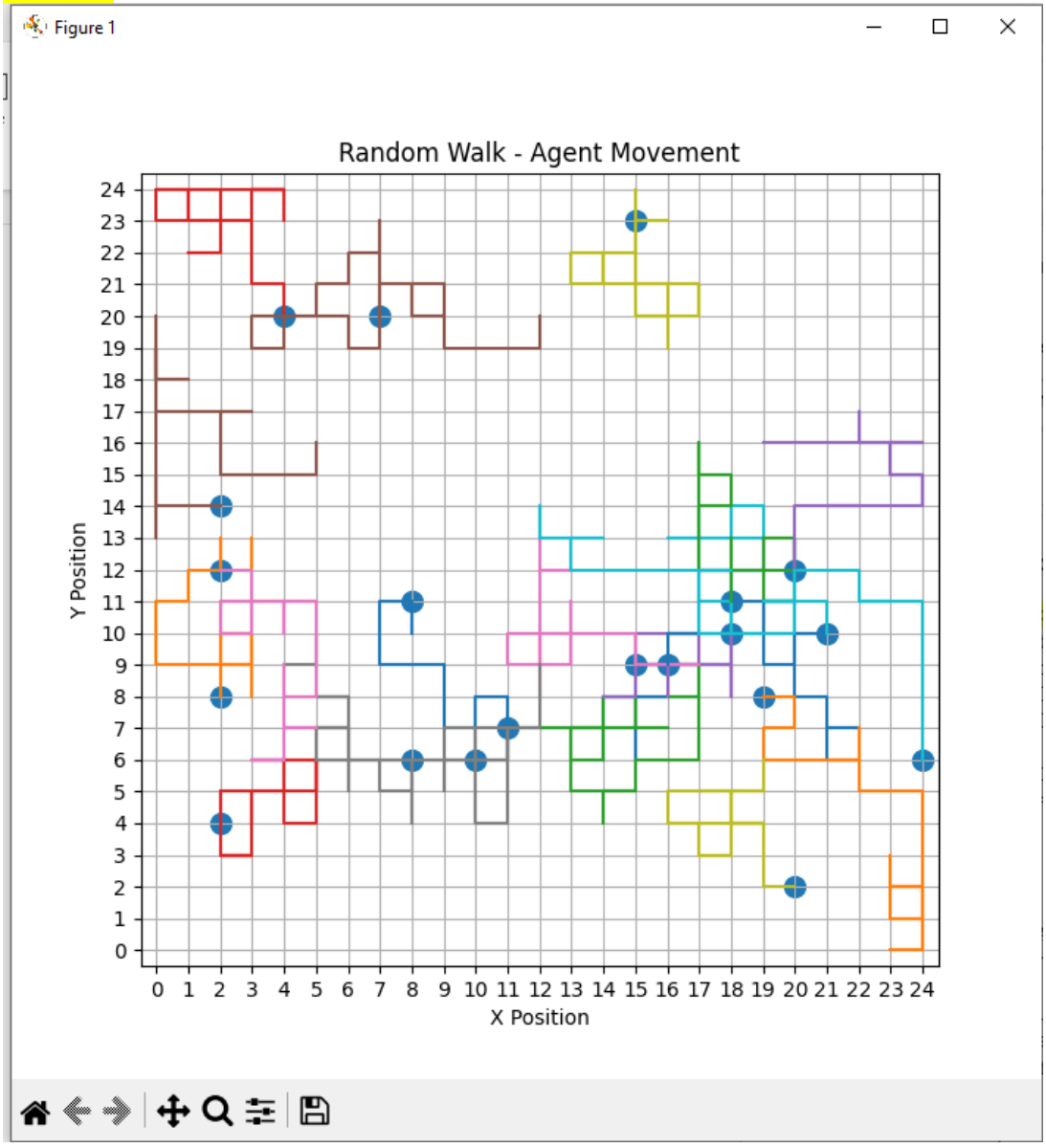

# -------- Start Animation --------
ani = animation.FuncAnimation(
    fig,
    update,
    frames=num_steps,
    interval=300,
    repeat=False
)

plt.show()
```

```
# -------- Display Final Positions --------
print("\n--- Final Positions ---")

for i, agent in enumerate(model.agents):
    print(f"Agent {i + 1}: {agent.position}")
```

Output



```
PS C:\Users\Dom> python -u "d:\Feu Alabangseu
Enter number of agents: 20
Enter grid size (e.g., 10 for 10x10): 25
Enter number of steps: 30

--- Final Positions ---
Agent 1: (8, 11)
Agent 2: (2, 8)
Agent 3: (11, 7)
Agent 4: (2, 4)
Agent 5: (18, 10)
Agent 6: (2, 14)
Agent 7: (15, 9)
Agent 8: (8, 6)
Agent 9: (15, 23)
Agent 10: (24, 6)
Agent 11: (16, 9)
Agent 12: (19, 8)
Agent 13: (18, 11)
Agent 14: (4, 20)
Agent 15: (20, 12)
Agent 16: (7, 20)
Agent 17: (2, 12)
Agent 18: (10, 6)
Agent 19: (20, 2)
Agent 20: (21, 10)
PS C:\Users\Dom>
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
5.

Answer the Questions ⇩

1. If the number of agents is very high compared to the grid size, it will become crowded and there will be fewer spaces to move.
2. Increasing the number of steps would allow the agents to move around the grid. This would allow them to move to a larger portion of the grid.
3. No the agents do not move outside the grid due to the movement rules provided by RandomWalker class. This rule check the grid boundaries before allowing an agent to move within the x and y coordinates.
4. A random – walk simulation can be used in real-life phenomena where movement occurs unpredictably such as diffusion of particles, movement of animals or spreading of objects in an environment.